Tema 2. Integridad de la Información

Dr. Rafael Ceballos Guerrero
Grupo de Investigación **Quivir**,
Tecnologías Inteligentes y de Seguridad de los Sistemas de Información
Departamento de Lenguajes y Sistemas Informáticos
Universidad de Sevilla





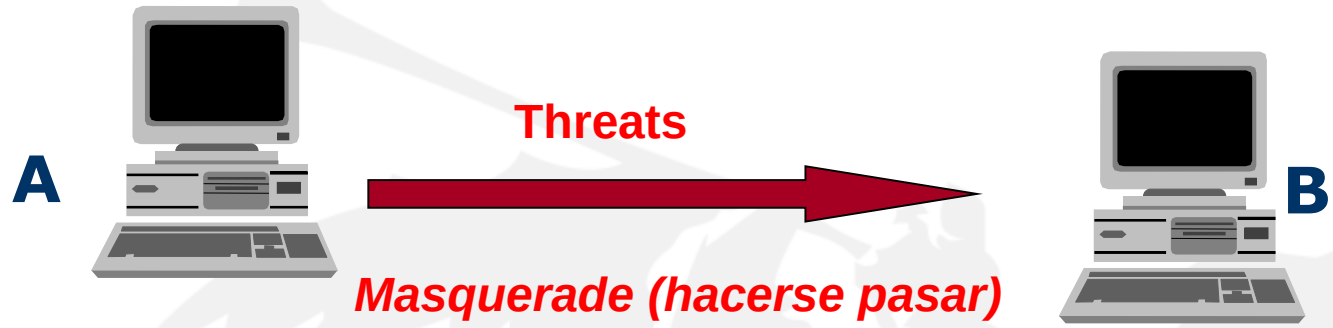
Implementación de Integridad en Transmission.
Amenazas



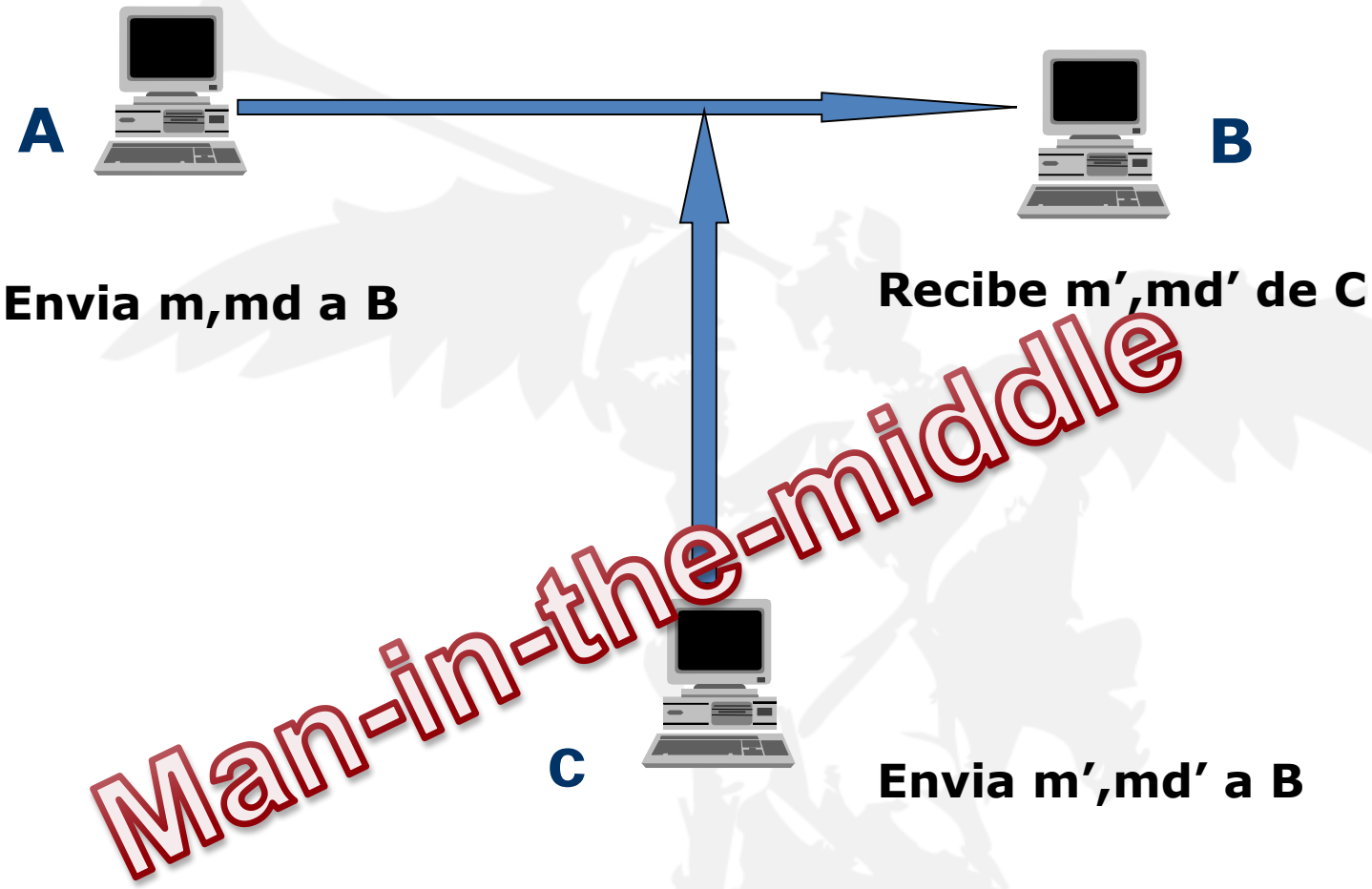
Envia m a B

¿Qué amenazas hay contra la integridad?

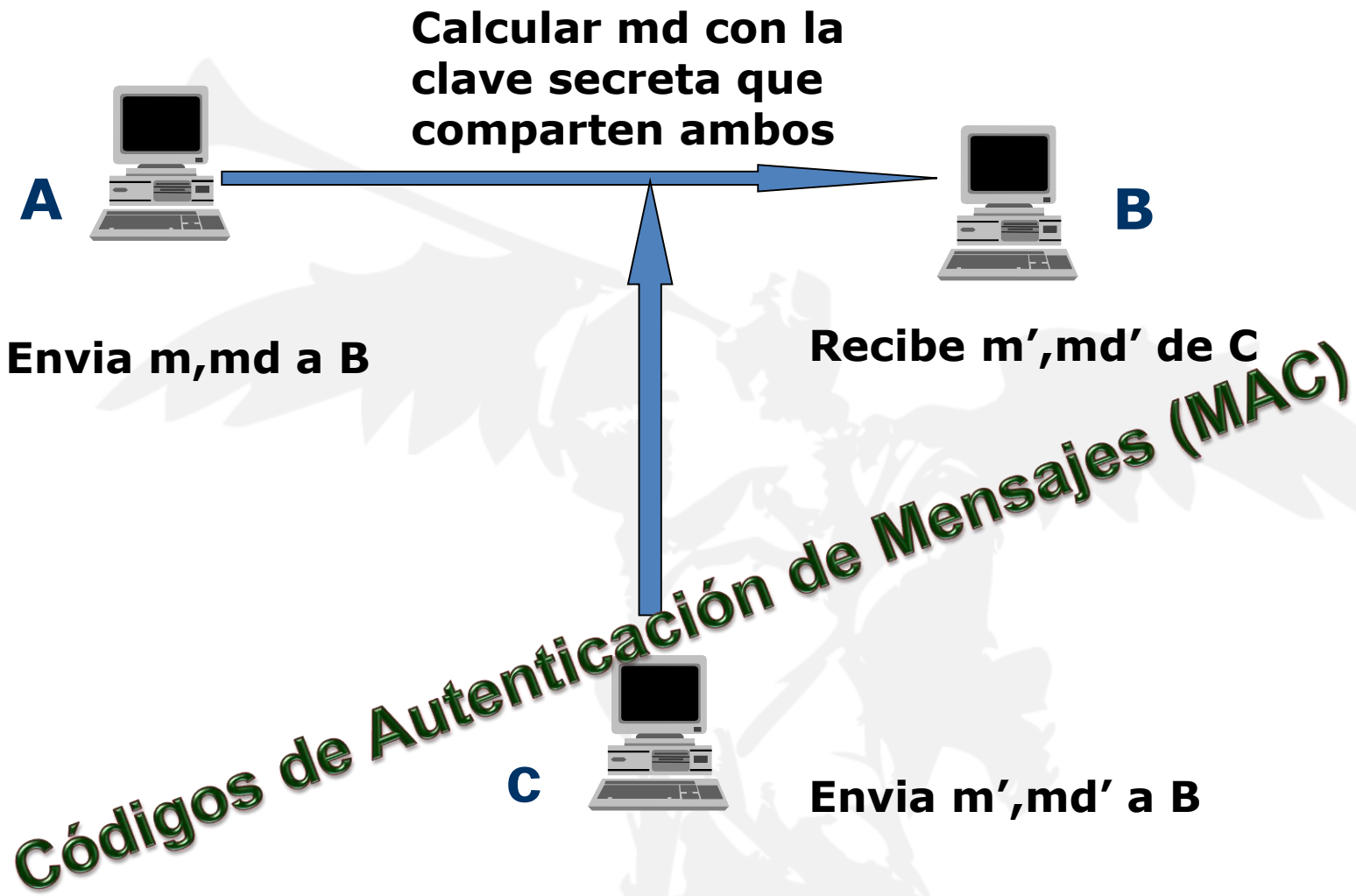
Implementación de Integridad en Transmisión.
Amenazas



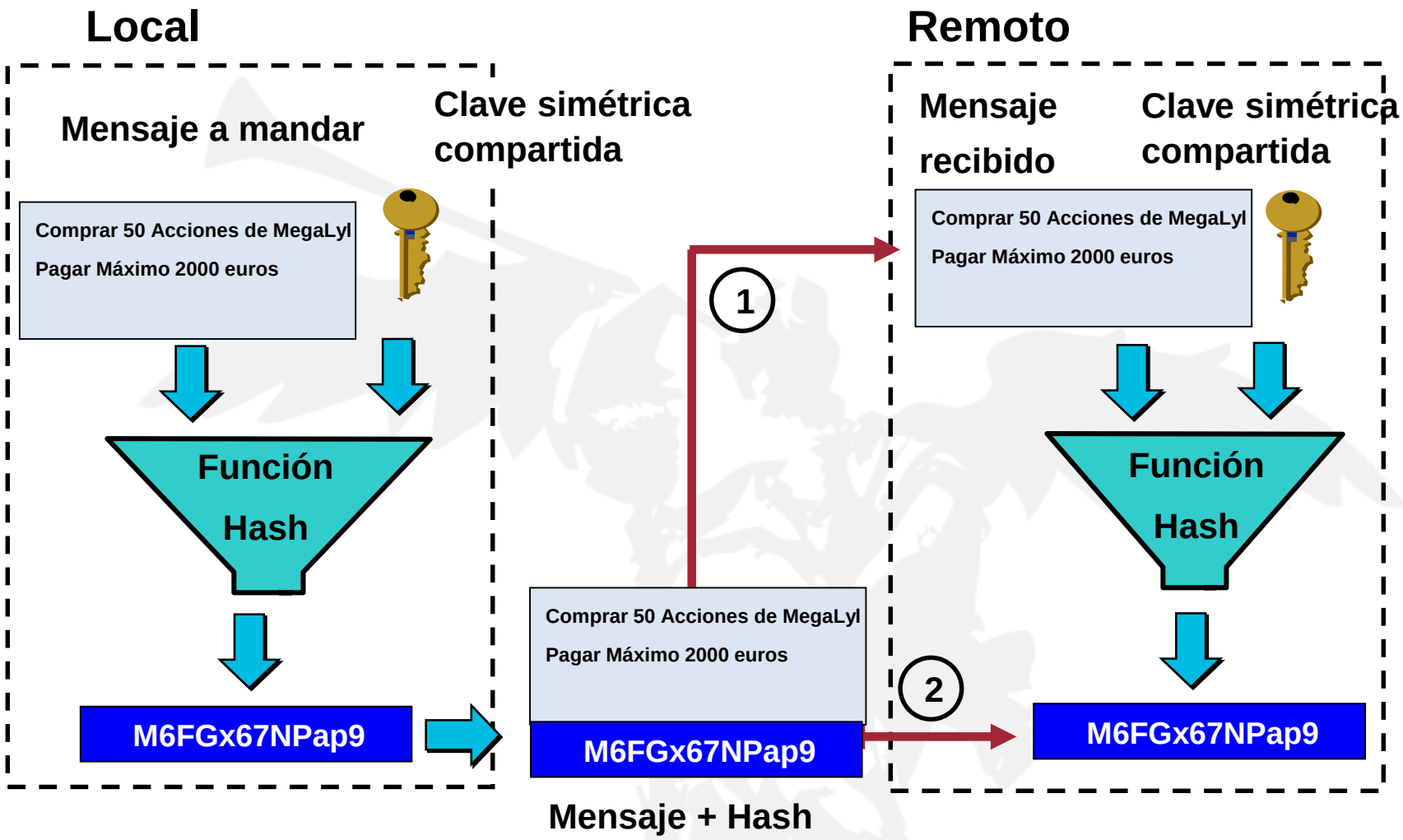
Implementación de Integridad en Transmisión.
Amenazas



Implementación de Integridad en Transmisión.
MAC



Implementación de Integridad en Transmisión. MAC



Implementación de Integridad en Transmisión
HMAC

- El resumen MAC utiliza un función hash y una clave que sólo conocen el emisor y receptor
  - El mensaje puede ser de cualquier tamaño
  - Permite comprobar si el mensaje fue alterado (integridad) y si proviene del remitente (no repudio)
- Existen diferentes grupos de funciones MAC: CBC-MAC, HMAC y UMAC
- HMAC se puede basar en MD5 (HMAC-MD5), SHA1 (HMAC-SHA1), SHA256 (HMAC-SHA256)

Implementación de Integridad en Transmisión
HMAC

- Algoritmo (Wikipedia)

```
function hmac (key, message)
    if (length(key) > blocksize) then
        key = hash(key) // llaves con más tamaño que el bloque se acortan
    end if
    if (length(key) < blocksize) then
        key = key ‖ [0x00 * (blocksize - length(key))] // llaves más cortas que el tamaño del
bloque se agrandan con ceros (donde ‖ es la concatenación)
    end if

    o_key_pad = [0x5c * blocksize] ⊕ key // Cuando el tamaño del bloque es el de la función
de hash subyacente
    i_key_pad = [0x36 * blocksize] ⊕ key // Donde ⊕ es el o exclusivo (XOR)

    return hash(o_key_pad ‖ hash(i_key_pad ‖ message)) // Donde ‖ es la concatenación
end function
```

Implementación de Integridad en Transmisión.
MAC en Java

- En java incluir los paquetes java.security.* y javax.crypto.*
  - La clase *Mac implementa los códigos de autenticación de mensajes, el constructor está protegido, los objetos se crean con*

    mac1= Mac.getInstance("HmacMD5");
  - Clase para generar claves ***KeyGenerator***

    kg=KeyGenerator.getInstance("HmacMD5");

    SecretKey clave= kg.generateKey();
  - El paso siguiente es inicializar el objeto ***Mac*** con los parámetros de la clave y el algoritmo especificado

    mac1.init(clave)
  - Se actualiza el objeto ***Mac*** con los bytes del mensaje

    mac.update(s.getBytes());
  - Termina la operación de MAC y se devuelve la mac

    byte [] b=mac.doFinal();

Implementación de Integridad en Transmisión
Problemas

- **Replay attack**: Un atacante intercepta la información transmitida, podría continuar enviando peticiones a una compañía, una y otra vez bajo el mismo nombre e información de compra. ¿¿Cuál es la solución??

- **Principio de Horton:** no sólo debemos verificar el resumen (MAC), también cierta información que se haya incluido para generar el resumen MAC
  - Por ejemplo un sello de tiempo
  - O datos relativos al contexto (número de mensaje,...) no incluidos en el mensaje, pero si usados para el cálculo del resumen

- **Solución al Replay Attack**: Un ***nonce*** se usa para dar 'originalidad' a los mensajes, ya que cada ***nonce*** se usa una vez y solamente una vez.

- **Multiple forgery attacks:** Un atacante puede conseguir mensajes diferentes cuyo resumen coincida con el enviado.
  - Por ejemplo se deben evitar las técnicas vulnerables por **Ataque Meet-in-the-middle** .

Referencias bibliográficas y webgráficas

- Sobre especificaciones de hashing seguro
  - http://csrc.nist.gov/publications/PubsFIPS.html#fips180-4
- **Encyclopedia of Cryptography and Security** de H.C.A. Van Tilborg y S. JaJodia 2ª Edicion. 2011 (disponible en books.google.es)
- **Handbook of Information Security, Key Concepts, Infrastructure, Standards and Protocols** por Hossein Bidgoli 2006 (en books.google,es)